

CNN Training

Objective

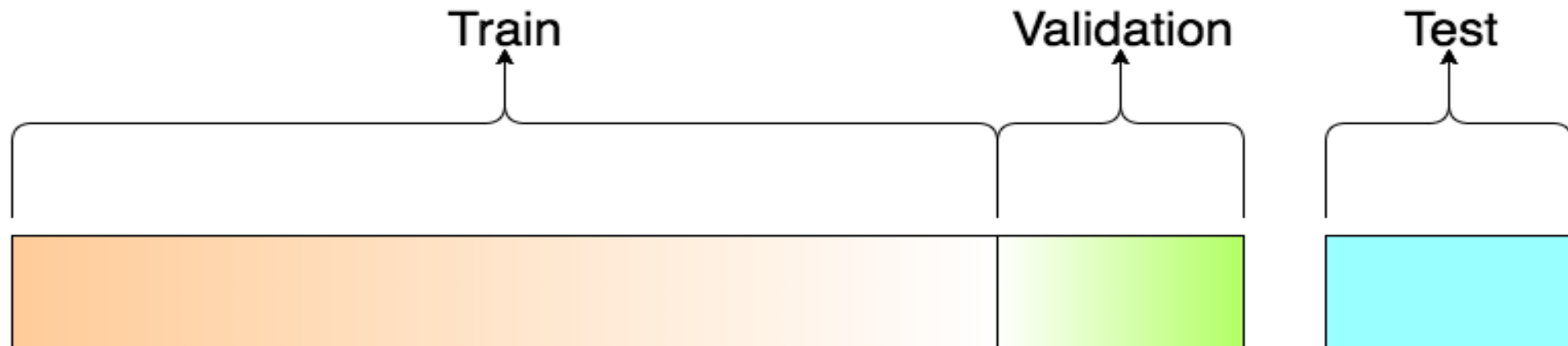
- When we train a model, we want its predictions to be as close as possible to the actual (true) values in the dataset.

Training

- Data
- What is Hyper-parameter Tuning?
- **How It Works (Step-by-Step)**
- **Loss Function**

Training

- **Data**
 - Training
 - Validation
 - Testing



Train Data

- **Training data** is the subset of the dataset used to train the model. This is where the model learns patterns, relationships, and features of the data. During training, the algorithm adjusts its parameters based on this data to minimize error and improve its predictions.
- **Purpose:** To fit the model.
- **Process:** The model iteratively adjusts its parameters to better fit the training data, using algorithms like gradient descent.

Validation Data

- **Validation data** is used to tune the model's hyper-parameters and to provide an unbiased evaluation of the model while tuning. Hyper-parameters are the aspects of the model that are not learned from the data but are set before the training process begins (e.g., learning rate, number of layers in a neural network).
- **Purpose:** To evaluate the model during training and assist in hyper-parameter tuning.
- **Process:** After each training iteration (or epoch), the model's performance is evaluated on the validation set. This helps to prevent overfitting to the training data.

Test Data

- **Test data** is the subset of the dataset used to provide an unbiased evaluation of a final model fit on the training dataset. It is only used after the model has been trained (and validated).
- **Purpose:** To assess the final model's performance and generalization to unseen data.
- **Process:** Once the model has been trained and hyper-parameters tuned, the test data is used to evaluate the model's performance. This step is crucial to understand how the model will perform in the real world.

Epochs, Batch

- Training a model for a single **epoch** usually isn't enough to capture all the patterns in the data. By repeating the process for **multiple epochs**, the model gradually learns and improves its performance.
- A **batch** refers to a **subset** of the training data that is processed together during **one forward pass** and **one backward pass** of the neural network. Since the entire dataset is often too large to process all at once, it is divided into smaller groups called **batches**.
- **Batch size** is a **hyper-parameter** that defines the number of training examples included in one batch. It determines how much data is processed before the model updates its weights

What is Hyper-parameter Tuning?

Hyper-parameters are configuration variables external to the model that control the training process.

Examples include:

- Number of filters in convolutional layers
- Number of neurons in dense layers
- Learning rate
- Batch size
- Number of epochs

How It Works (Step-by-Step)

- **Initialize parameters** (weights) randomly.
- **Make predictions** using current weights.
- **Compute the loss** (error).
- **Calculate the gradient** (the direction of steepest increase in loss).
- **Update weights in the opposite direction** of the gradient.

How It Connects to Training

1. Model makes predictions...
- 2. Loss function calculates error**
- 3. Gradient descent updates weights**
4. Loss decreases
5. Model improves

Why Do We Need a Loss Function?

- When training a model, we need a way to:
 - Measure error
 - Quantify performance
 - Guide improvement

Loss Function

- A loss function is a mathematical formula that measures how wrong a model's prediction is. It gives us a number that represents the error between:
 - The true value (actual answer)
 - The predicted value (model output)
- **Simple Example**
 - Actual house price = \$200,000
 - Model prediction = \$180,000
 - The model is off by \$20,000.
- The loss function calculates this error mathematically.

Loss function calculates error

To measure how far off the predictions are, we use a **loss function**, such as:

- Mean Squared Error (MSE) for regression
- Cross-Entropy Loss for classification

The goal of training is to **minimize this loss**.

Common Loss Functions

- **Mean Squared Error (MSE) – (Used in Regression)**

$$MSE = \frac{1}{n} \sum (y_{true} - y_{pred})^2$$

For a single sample:

$$L = (y - \hat{y})^2$$

Where:

- y = true value
 - $\hat{y}$ = predicted value
- Takes the difference
 - Squares it (so negatives don't cancel positives)
 - Averages over all data points

Loss function calculates error:(MSE)

- **Regression problems examples**

- Predicting house prices
- Temperature forecasting
- Stock values

Sometimes used in:

- Binary classification (but not ideal)

Cross-Entropy Loss

- Cross-Entropy measures the difference between:
 - True probability distribution
 - Predicted probability distribution
- Types
 - Binary Cross-Entropy (BCE)
 - Categorical Cross-Entropy

Cross-Entropy Loss

Binary Cross-Entropy (BCE) When Used

- Spam detection
- Disease prediction (yes/no)
- Fraud detection

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Where:

- $y \in \{0, 1\}$
- $\hat{y}$ = predicted probability (from sigmoid)



Categorical Cross-Entropy

Cross-Entropy is Better for Classification

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

Where:

- C = number of classes
- y_i = one-hot encoded true label
- $\hat{y}_i$ = predicted probability (from softmax)

learning rate

Learning rate controls how strongly the model reacts to its mistakes.

- The **learning rate** is a small number that controls **how big a step the model takes when updating its weights** during training.
- The learning rate is one of the most important **hyper-parameters** because it directly affects:
 - Speed of training
 - Stability
 - Final model accuracy

What is Gradient Descent?

- **Gradient Descent** is an optimization algorithm used to **minimize the loss function** by adjusting the model's parameters (**weights**).
- Think of it like this:
 - Imagine the loss function as a hill.
 - The top of the hill = high error.
 - The bottom of the hill = lowest error (best fit).
 - Gradient descent helps us **move downhill step by step** until we reach the minimum.

Types of Gradient Descent

- **Batch Gradient Descent** – Uses entire dataset
- **Stochastic Gradient Descent (SGD)** – Uses one example at a time
- **Mini-batch Gradient Descent** – Uses small batches (most common in deep learning)

Learning Rate (α): Gradient Descent

- The **learning rate** is a small number that controls **how big a step the model takes when updating its weights** during training.
- The **learning rate (α)** controls **how big a step** the model takes when updating weights during gradient descent.

- w = weight
- α = learning rate (step size)
- L = loss function
- $\frac{\partial L}{\partial w}$ = gradient

$$w = w - \alpha \frac{\partial L}{\partial w}$$

Learning Rate (α):

- The **learning rate (α)** controls how much we **adjust weights during backpropagation.**
- **Why It's Critical in CNNs**
 - Many layers
 - Thousands to millions of parameters
 - Complex loss surfaces
- If learning rate is:
- **Too high** → exploding updates, unstable training
- **Too low** → very slow convergence

Optimizer

- Learning rate controls how big each correction is.
- The optimizer decides how those corrections are calculated and applied.
- Types
 - Stochastic Gradient Descent (SGD) is the most basic optimizer.
 - AdaGrad is an optimizer that adjusts the learning rate for each parameter based on its gradients in the past.
 - Adam is an adaptive learning rate optimizer that combines the ideas of SGD and AdaGrad. It adjusts the learning rate for each parameter based on the gradient's first and second moments.

code

- **Torch**

```
import torch.optim as optim

optimizer = optim.Adam(model.parameters(), lr=0.001)
```

- **tensorflow**

```
from tensorflow.keras.optimizers import Adam

optimizer = Adam(learning_rate=0.001)

model.compile(optimizer=optimizer, loss='categorical_crossentropy')
```

Interpreting the Learning Curves

- Under-fitting:**

Training line is far from data → never converges to target → high bias.

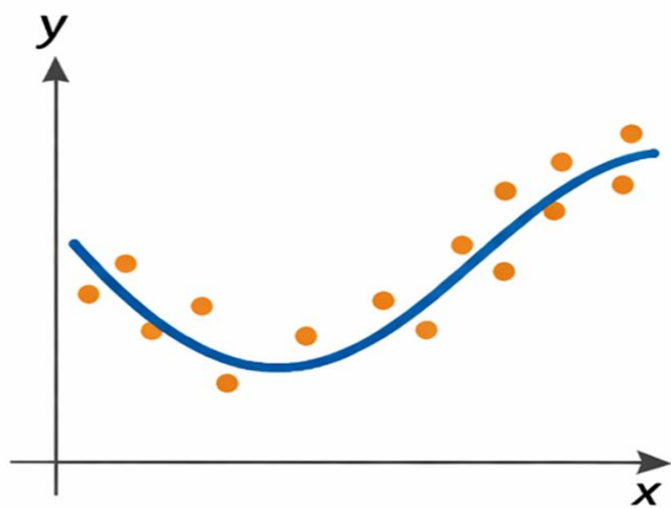
- Good fit / proper convergence:**

Training line captures the trend → converges smoothly → low bias & variance.

- Over-fitting / overshooting analog:**

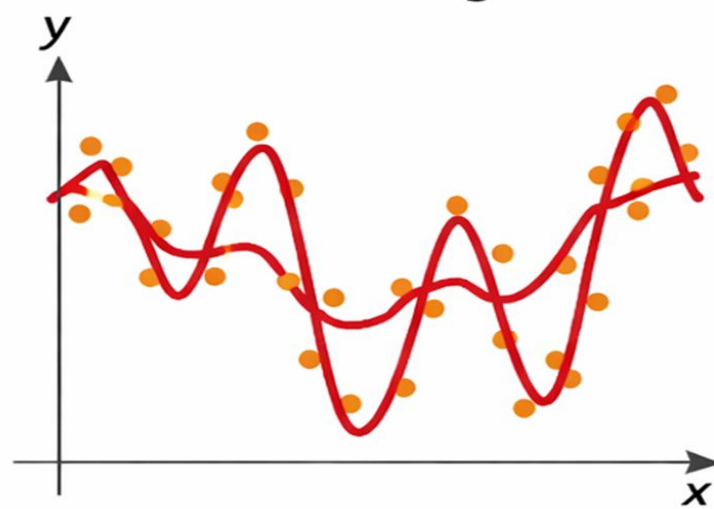
Model “jumps around” trying to pass through every point → oscillates → fits noise → high variance.

Good Fit



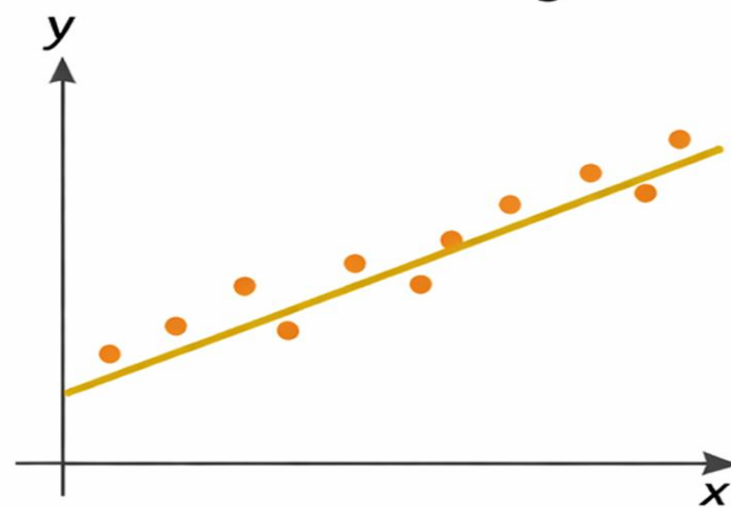
Good Fit

Overfitting



Overfitting

Underfitting



Underfitting

Interpreting the Learning Curves

- An **over-fit model**

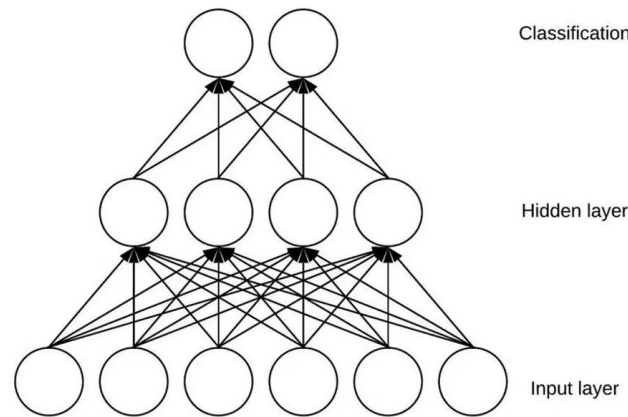
- can result in high model accuracy on training data but low accuracy on new data due to memorization instead of generalization.
- Overfitting happens when engineers use a machine learning model with too many parameters or layers
- When trained on a small or noisy data set, the model risks memorizing specific data points and noise rather than learning the general patterns.

- An **under-fit model**

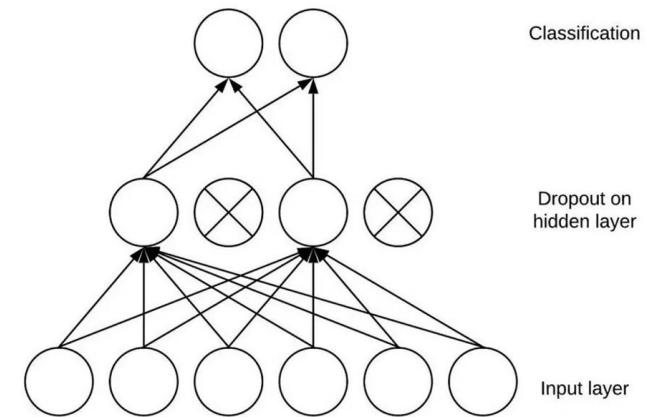
- performs poorly on training data and testing data because it fails to capture the dominant patterns in the data set.
- Engineers typically identify under-fitting through consistently poor performance across both data sets.
- Under-fit models also tend to show high errors in learning curves

Drop out

- **Dropout** is a **regularization technique** used in neural networks to reduce **overfitting**.
- During training, dropout **randomly turns off (drops)** a percentage of neurons in a layer at each iteration.



Without Dropout



With Dropout

- **Accuracy:** $\frac{TP + TN}{TP + TN + FP + FN}$
- **Precision:** $\frac{TP}{TP + FP}$
- **Recall (Sensitivity):** $\frac{TP}{TP + FN}$
- **F1-Score:** $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

EVALUATING DEEP LEARNING MODELS

True Positive	False Positive
False Negative	True Negative



Accuracy



Precision



F1-Score



F1-Score